

PROgramming in LOGic

CS 331 Tutorial # 3

Rashmi Dutta Baruah

Department of Computer Science & Engineering

IIT Guwahati

Outline

- Input and Output
- Meta-predicates
- Meta-interpreters

Input and output

- Input and output (other than that associated with querying the program) is done using built-in procedures.
- Input files – input streams, Output files – output streams
- The user terminal is treated as just another stream (file) called **user**.
- Switching between streams is done by:

see (File) File becomes the current input stream

tell (File) File becomes the current output stream

seen close the current input stream

told close the current output stream

Example

write_example :-

```
tell('output.txt'),    % Open file for writing
write_on_file('Hello Prolog'), nl,
told.                  % Close output stream
```

read_example :-

```
see('output.txt'),    % Open file for reading
read_from_file(user_input, Line),
write(Line), nl,
seen.                 % Close input stream
```

Input and Output

- Files are read and written in two ways:
 - as sequences of characters
 - as sequences of terms
- Built-in procedures for reading and writing characters and terms are:
 - **read(Term)** - input next term
 - **write(Term)** - output Term
 - **put(CharCode)** - output character with the given ASCII code
 - **get0(CharCode)** - input next character
 - **get(CharCode)** - input next 'printable' character
- Two procedures help formatting:
 - **nl** - output new line
 - **tab(N)** - output N blanks

Example

```
write_example(X) :-  
    tell('output.txt'),      % Open file for writing  
    write_on_file(X), nl,  
    told.                    % Close output stream  
write_on_file(X) :-  
    read(X), write(X) .
```

```
read_example(X) :-  
    see('output.txt'),      % Open file for reading  
    read_from_file(X),  
    seen.                   % Close input stream  
read_from_file(X) :-  
    read(X) .
```

Example

```
cube(N,C):-  
    C is N*N*N.
```

```
cube:-  
    read(X),process(X).  
process(stop):- !.  
process(N):-  
    C is N*N*N,  
    write(C),  
    cube.
```

```
cube:-  
    write('Next item, please:  
' ),  
    read(X),  
    process(X).  
process(stop):- !.  
process(N):-  
    C is N*N*N,  
    write('Cube of'),write(N),  
    write('is'), write(c),  
    nl, cube.
```

Constructing and Decomposing Atoms

- The procedure **name(Atom, Codelist)** decomposes and constructs atoms. Codelist is the list of ASCII code of the characters in Atom
- **name** predicate relates atoms and their ASCII encodings
- There are two uses of name:
 - Given an atom, break it down into single characters
 - Given a list of characters, combine them into an atom

```
?- name(hello, L).  
L = [104, 101, 108, 108, 111].
```

```
?- name(X, [112,114,111,108,111,103]).  
X = prolog.
```

Example

- Check whether the first two letters of a car number plate are AS (e.g., AS01AB1234).

```
starts_with_as(NumberPlate) :-  
    name(NumberPlate, [_65,83|_]).
```

```
?- starts_with_as('AS01AB1234').  
true.
```

```
?- starts_with_as('DL01AB1234').  
false.
```

Meta-Predicates

- **Meta-predicates** allow to match, query , and manipulate other predicates.
- They can be used to write **Meta-programs** : programs that take other programs as input.
- Some of the meta-predicates:

```
asserta(X) and assertz(X)
retract(X)
clause(X, Y)
listing(A)
call(X)
functor(T, F, N)
arg(N, T, A)
name(A, L)
```

Meta-Predicates

- **asserta(X)** and **retract(X)** : used for database manipulation
- **clause(X,Y)** : X and Y are matched with the head and body of an existing clause. X must be instantiated such that the main predicate of the clause is known.

```
append([],X,X) .  
append([A|B],C,[A|D]):- append(B,C,D) .  
?- clause(append(X1,X2,X3),Y) .  
X1 = [],  
X2 = X3,  
Y = true ;  
X1 = [_G2269|_G2270],  
X3 = [_G2269|_G2273],  
Y = append(_G2270, X2, _G2273) .
```

Meta-Predicates

- **call(X)** : it succeeds if x succeeds, and fails if x fails.
- **functor(S,F,N)** : if S is a structure or an atom, F is matched with the functor and N is matched with arity (number of arguments). It fails if S is not a structure or an atom.

```
?- functor(date(dd,mm,yy),F,N) .  
F = date,  
N = 3.  
?- functor(date,F,N) .  
F = date,  
N = 0.
```

- **arg(N,S,A)** : used to access a particular argument of structure S specified by N .

```
?- arg(2,related(john,mother(jane)),X) .  
X = mother(jane) .
```

Meta-Predicates

- **listing(X)** : lists all the clauses with `x` as predicate.
- **name(A,L)** : characters for the atom `A` are members of the list `L`. It can be used either to find the characters for a given atom, or find an atom that has some given characters.

```
?- name(apple,X).  
X = [97, 112, 112, 108, 101].
```

Meta-interpreters

- Meta-interpreter : Meta-program
 - It is an interpreter for the language written in the language itself.

```
% A meta-interpreter for building a proof tree
```

```
solve(true,true) :- !.
```

```
solve((A,B),(ProofA,ProofB)):-
```

```
    !,solve(A,ProofA),solve(B,ProofB).
```

```
solve(A,(A -> Proof)):-
```

```
    clause(A,B),solve(B,Proof).
```

Meta-interpreters

```
male(albert).  
male(edward).  
female(alice).  
female(Jane).  
parents(edward,Jane,albert).  
parents(alice,Jane,albert).  
  
sister_of(X,Y):- female(X),  
                  parents(X,M,F),  
                  parents(Y,M,F).
```

sister_of(alice,edward)

female(alice)

true

parents(alice,Jane,albert)

true

parents(edward,Jane,albert)

true

```
?- solve(sister_of(alice,edward),Proof).  
Proof = (sister_of(alice, edward)-> (female(alice)-  
>true), (parents(alice, Jane, albert)->true),  
(parents(edward, Jane, albert)->true)).
```

Summary

- In this tutorial we learnt:
 - Input-ouput
 - some more built-in predicates that are meta-predicates
 - meta-interpreters with an example